

신호 모델 엣지 배치: INT8 QAT 대 PTQ

양자화가 되는 모델과 안 되는 모델, 실측 비교

2i

2026-07-29

초록

엣지에서 신호 모델을 돌리려는 곳은 대개 GPU가 없다. 온프레임 서버의 저가 CPU, 임베디드 보드, 라우터급 하드웨어가 실제 배치 대상이고, 예산은 클라우드 GPU 추론 대비 훨씬 작다. 이런 환경에서 모델을 INT8로 양자화하면 크기와 지연이 동시에 줄어든다는 것은 잘 알려진 사실이다. 그러나 '양자화하면 공짜로 3배 작아진다'는 통념은 정확도라는 대가를 자주 숨긴다. 이 논문은 작은 자동변조분류(AMC) CNN 하나를 대상으로 두 가지 INT8 양자화 경로, 즉 학습 후 양자화(PTQ, calibration만 거치고 재학습 없음)와 양자화 인식 학습(QAT, 양자화를 학습 루프 안에 넣고 미세조정)을 같은 데이터, 같은 모델, 같은 측정 하니스로 비교했다. 결과는 단순하고 극단적이다. FP32 기준 정확도 93.18퍼센트에서 PTQ는 61.91퍼센트로 무너졌다(31.27 퍼센트포인트 손실). 같은 모델에 15 에폭의 양자화 인식 미세조정을 붙인 QAT는 91.45퍼센트를 회복했다(1.73퍼센트포인트 손실). 두 경로 모두 모델 크기를 119.3킬로바이트에서 35.5킬로바이트로 똑같이 70.3퍼센트 줄이고, 추론 지연도 배치 크기 1 기준으로 43퍼센트 이상 줄인다. 즉 크기·지연 이득은 두 경로가 동일하게 받고, 정확도라는 값을 지불하는 방식만 다르다. PTQ는 그 값을 거의 전부 잃고, QAT는 그 값의 94.5퍼센트를 되찾는다. 결론은 명확하다. 저가 온프레임 엣지에 신호 분류 모델을 INT8로 배치하려면 PTQ 단독으로는 실전에 쓸 수 없고, QAT를 기본 경로로 채택해야 한다.

목차

1 요약	3
2 왜 엣지 양자화인가	3
3 PTQ 대 QAT: 두 가지 양자화 경로	3
4 데이터와 실험 설정	4
5 결과	4
6 한계	5
7 재현	6
8 참고문헌	6

1 요약

엣지에서 신호 모델을 돌리려는 곳은 대개 GPU가 없다. 온프레임 서버의 저가 CPU, 임베디드 보드, 라우터급 하드웨어가 실제 배치 대상이고, 예산은 클라우드 GPU 추론 대비 훨씬 작다. 이런 환경에서 모델을 INT8로 양자화하면 크기와 지연이 동시에 줄어든다는 것은 잘 알려진 사실이다. 그러나 “양자화하면 공짜로 3배 작아진다”는 통념은 정확도라는 대가를 자주 숨긴다. 이 논문은 작은 자동변조분류(AMC) CNN 하나를 대상으로 두 가지 INT8 양자화 경로, 즉 학습 후 양자화(PTQ, calibration만 거치고 재학습 없음)와 양자화 인식 학습(QAT, 양자화를 학습 루프 안에 넣고 미세조정)을 같은 데이터, 같은 모델, 같은 측정 하니스로 비교했다. 결과는 단순하고 극단적이다. FP32 기준 정확도 93.18퍼센트에서 PTQ는 61.91퍼센트로 무너졌다(31.27 퍼센트포인트 손실). 같은 모델에 15 에폭의 양자화 인식 미세조정을 붙인 QAT는 91.45퍼센트를 회복했다(1.73퍼센트포인트 손실). 두 경로 모두 모델 크기를 119.3킬로바이트에서 35.5킬로바이트로 똑같이 70.3퍼센트 줄이고, 추론 지연도 배치 크기 1 기준으로 43퍼센트 이상 줄인다. 즉 크기·지연 이득은 두 경로가 동일하게 받고, 정확도라는 값을 지불하는 방식만 다르다. PTQ는 그 값을 거의 전부 잃고, QAT는 그 값의 94.5퍼센트를 되찾는다. 결론은 명확하다. 저가 온프레임 엣지에 신호 분류 모델을 INT8로 배치하려면 PTQ 단독으로는 실전에 쓸 수 없고, QAT를 기본 경로로 채택해야 한다.

2 왜 엣지 양자화인가

무선 신호 분류, 간섭 탐지, 기기 지문 인식 같은 작업을 클라우드 GPU 위에서 돌리는 것은 연구 단계에서는 편리하지만, 실제 배치 지점에서는 대개 선택지가 아니다. 통신 장비 안에 들어가는 추론 모듈은 GPU를 엿을 전력 예산도, 공간도 없는 경우가 많다. 보안 요구사항 때문에 데이터를 외부로 보낼 수 없어 온프레임 서버에서 돌려야 하는 경우도 흔하다. 이때 서버는 클라우드 GPU 인스턴스가 아니라 저가 CPU 서버이거나, 라우터에 내장된 저전력 프로세서다. 이런 하드웨어에서 부동소수점 32비트(FP32) 모델을 그대로 돌리면 메모리 점유가 크고 추론 지연도 길다.

INT8 양자화는 이 문제에 대한 표준적인 답이다. 가중치와 활성화값을 32비트 부동소수점이 아니라 8비트 정수로 표현하면 모델 크기가 이론상 4배 줄고, 정수 연산 커널을 지원하는 CPU에서는 부동소수점 연산보다 빠르게 실행된다. 문제는 이 이득이 공짜가 아니라는 점이다. 부동소수점 값을 8비트 정수로 매핑하는 과정에서 표현 정밀도가 줄어들고, 이 정밀도 손실이 모델 정확도에 어떤 영향을 미치는지는 모델과 데이터마다 다르다. 큰 언어모델이나 이미지 분류 CNN에서는 INT8 PTQ가 정확도 손실 없이 잘 통하는 사례가 많이 보고되어 있어서, 이 낙관이 신호 모델에도 그대로 옮겨질 것이라고 가정하기 쉽다. 이 논문은 그 가정을 검증한다.

신호 모델은 이미지 분류 CNN과 다른 점이 있다. 입력이 IQ(동위상-직교위상) 시계열이고, 배치 초기 단계의 배치정규화(BatchNorm) 통계가 신호 스케일을 안정화하는 데 핵심적인 역할을 한다. PTQ의 표준 경로는 배치정규화를 컨볼루션에 접어 넣고(fold) 그 뒤에 활성화값 스케일을 캘리브레이션 데이터로 고정하는데, 이 순서가 신호 모델에서는 특히 취약할 수 있다는 것이 이 실험의 핵심 가설이었다. 아래에서 그 가설을 실측으로 검증한다.

3 PTQ 대 QAT: 두 가지 양자화 경로

두 경로는 정수 연산 커널로 수렴한다는 점에서 목적지는 같지만, 거기에 도달하는 방식이 다르다.

PTQ(post-training quantization, 학습 후 양자화)는 이미 학습이 끝난 FP32 모델을 그대로 가져와 (1) 컨볼루션, 배치정규화, ReLU를 하나의 융합 모듈로 접고(fuse), (2) 관찰자(observer)를 끼운 뒤 검증 데이터 일부를 통과시켜 각 층의 활성화값 분포를 캘리브레이션하고, (3) 그 통계로 스케일과 제로포인트를 확정해 정수 커널로 변환한다. 재학습이 전혀 없다는 것이 핵심이다. 배치 하나만 흘려보내면 몇 초 안에 끝나는 값싼 경로다.

QAT(quantization-aware training, 양자화 인식 학습)는 같은 FP32 체크포인트에서 시작하지만, 양자화를 시뮬레이션 하는 가짜양자화(FakeQuantize) 연산을 순전파에 끼운 채로 몇 에폭을 추가로 미세조정한다. 이 실험에서는 배치정규화를 즉시 접어 얼려버리지 않고, 학습 중에는 계속 갱신되는 융합 모듈(ConvBnReLU1d intrinsic module)로 유지했다. 그리고 미세조정 후반부에 관찰자를 먼저 동결하고, 그다음에 배치정규화 이동통계를 동결하는 표준 안정화 순서를 따랐다. 이 순서를 지키지 않으면 학습이 수렴하지 않고 발산하거나 진동한다는 것이 실측에서도 확인됐다(4절 참고). 미세조정이 끝나면 PTQ와 동일하게 정수 커널로 변환한다.

두 경로 모두 최종적으로는 같은 종류의 정수 연산 커널을 쓴다. 즉 크기와 지연 이득의 상한은 두 경로가 원칙적으로 동일하다. 차이는 오직 정확도를 얼마나 지키느냐에 있다. QAT가 유일하게 추가로 지불하는 비용은 학습 파이프라인에 미세조정 단계를 몇 에폭 더 붙이는 시간이다.

4 데이터와 실험 설정

데이터는 공개된 무선 신호 벤치마크 RadioML2016.10a의 6dB 신호대잡음비(SNR) 부분집합을 썼다. IQ 시계열(2채널, 길이 128) 형태의 11개 변조 클래스(8PSK, AM-DSB, AM-SSB, BPSK, CPFSK, GFSK, PAM4, QAM16, QAM64, QPSK, WBFM) 분류 과제다. 학습에 8,415개, 검증(조기종료용)에 1,485개, 테스트에 1,100개 샘플을 썼다. 테스트 세트는 학습과 QAT 미세조정 양쪽 어디에도 사용하지 않아 데이터 누수가 없다.

모델은 Conv1d 3개 블록(각 블록이 배치정규화와 ReLU를 포함)과 전역평균풀링, 완전연결 2개 층으로 구성된 소형 CNN이다. 파라미터 수는 수만 개 수준으로, 엣지 배치를 염두에 둔 크기다. FP32 기준선은 80에폭(조기종료 patience=20)을 학습해 검증 정확도 92.996퍼센트, 테스트 정확도 93.18퍼센트에 도달했다. 학습에는 라벨 스무딩 0.05를 적용했다.

PTQ는 이거 모델(fuse_modules로 컨볼루션+배치정규화+ReLU 융합) 위에 eager-mode 정적 양자화 API를 적용했다. 검증 세트로 관찰자를 캘리브레이션한 뒤 정수 커널로 변환했다. 백엔드는 qnnpack으로, Apple Silicon을 포함한 ARM 계열 CPU에 네이티브로 최적화된 양자화 엔진이다.

QAT는 fuse_modules_qat로 컨볼루션+배치정규화+ReLU를 학습 가능한 융합 모듈로 묶고, 가중치와 활성화값에 FakeQuantize 관찰자를 끼운 채로 학습률 3e-5(FP32 학습률의 10분의 1)로 15에폭을 미세조정했다. 11에폭째부터 관찰자를 동결하고, 13에폭째부터 배치정규화 이동통계를 동결한 뒤 나머지 에폭을 마쳤다. 이후 정수 커널로 변환했다.

측정은 세 항목이다. 정확도는 테스트 세트에서 argmax 예측과 라벨을 직접 비교해 계산했다. 모델 크기는 state_dict를 실제로 직렬화했을 때의 바이트 수다. 추론 지연은 배치 크기 1, CPU 실행 기준으로 워밍업 30회 후 300회 실제 순전파를 측정해 p50과 p95를 기록했다. 모든 수치는 모델이 스스로 보고한 값이 아니라 측정 코드가 직접 계산한 값이다.

5 결과

아래 표는 같은 실행 안에서 FP32 기준선, PTQ, QAT를 동일한 테스트 세트와 동일한 측정 하니스로 비교한 값이다.

구성	테스트 정확도	정확도 변화	모델 크기	크기 변화	p50 지연	p50 변화
FP32 (기준선)	93.18%	-	119.3 KB	-	0.278 ms	-
INT8 PTQ (calibration 만)	61.91%	-31.27%p	35.5 KB	-70.3%	0.158 ms	-43.2%
INT8 QAT (15 에폭 미세조정)	91.45%	-1.73%p	35.5 KB	-70.3%	0.155 ms	-44.2%

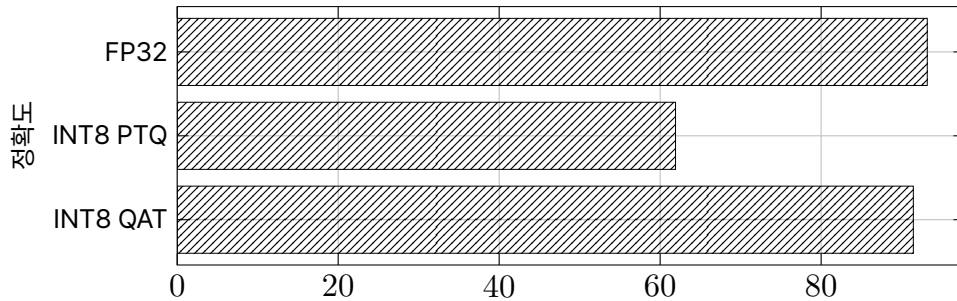


그림 1: 테스트 정확도 비교

크기·지연 이득은 PTQ와 QAT가 사실상 동일하다는 점이 이 실험의 첫 번째 결론이다. 두 경로 모두 같은 qnnpack 정수 커널로 변환되기 때문에, 미세조정 단계가 추론 그래프 자체를 바꾸지는 않는다. QAT는 오히려 p50 지연이 0.155밀리초로 PTQ의 0.158밀리초보다 약간 더 낮게 측정됐다(측정 잡음 범위 내이지만, 최소한 QAT가 지연 이득을 희생하지 않는다는 것은 명확하다).

두 번째, 그리고 이 논문의 핵심 결론은 정확도다. PTQ는 calibration만으로 정확도를 93.18퍼센트에서 61.91퍼센트로 떨어뜨렸다. 31.27퍼센트포인트라는 손실은 실전 배포에서 받아들이 수 있는 수준이 아니다. 같은 모델에 QAT를 적용하면 91.45

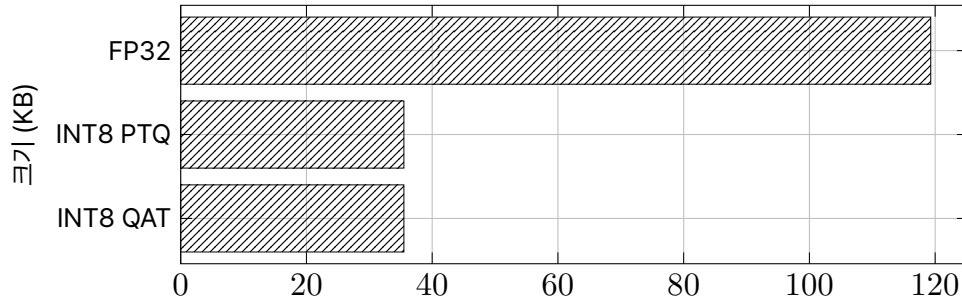


그림 2: 모델 크기 비교

퍼센트로 회복된다. 손실이 1.73퍼센트포인트로 줄어든다는 것은, PTQ가 낸 손실의 94.5퍼센트를 QAT가 되찾았다는 뜻이다 ($(31.27 - 1.73) / 31.27 = 0.945$). 남은 1.73퍼센트포인트는 무시할 수는 없지만, PTQ의 31.27퍼센트포인트와는 질적으로 다른 위험 등급이다.

QAT 학습 곡선을 들여다보면 왜 배치정규화 동결 순서가 중요한지가 드러난다. 미세조정 1에폭부터 10에폭까지 검증 정확도는 0.22에서 0.87 사이를 크게 진동했다. 관찰자가 아직 활성화 스케일을 계속 갱신하는 중이라 학습이 안정되지 않은 상태다. 관찰자를 동결한 11에폭에서도 진동이 이어지다가, 배치정규화 이동통계를 동결한 13에폭에서 검증 정확도가 0.90으로 급등하고, 15에폭에서 0.9111로 마무리됐다. 관찰자와 배치정규화를 순서대로 동결하는 절차 자체가 수렴의 전환점이었다는 뜻이다. 이 순서를 생략하면 이 회복이 안 나올 가능성이 높다(이 실험에서 직접 검증하지는 않았지만, 진동 패턴이 그 가설을 지지한다).

QAT의 추가 비용은 미세조정 시간이다. 이 모델 규모에서는 CPU 기준 15에폭에 90.8초가 걸렸다. FP32 기준선 학습(80에폭, 37.8초)보다 에폭당 훨씬 느린 것은 가짜양자화 연산의 오버헤드 때문이지만, 총 미세조정 시간 자체는 이 모델 규모에서는 무시할 만한 수준이다.

참고로 fp16(모델을 16비트로 캐스팅만 하고 재학습 없음)도 같은 실험에서 함께 측정했다. 정확도 손실은 0.18퍼센트포인트로 QAT보다도 작았지만, 크기 절감은 46.9퍼센트로 INT8보다 작고, 이 CPU 환경에는 네이티브 fp16 커널이 없어 지연이 오히려 4배 가까이(1.086밀리초) 늘었다. “16비트니까 더 빠르다”는 가정은 이 하드웨어에서는 성립하지 않았다. fp16은 이 실험의 주된 비교 대상은 아니지만, 정밀도를 낮추는 모든 선택이 곧 속도 이득을 뜻하지는 않는다는 것을 보여주는 반례로 남긴다.

6 한계

이 결과를 일반적인 결론으로 확대하기 전에 아래 경계를 명확히 해야 한다.

첫째, 이 결과는 이 자동변조분류 CNN, 그리고 6dB SNR이라는 특정 조건에 국한된다. 더 크고 구조가 다른 모델, 예를 들어 어텐션 통계 풀링을 쓰는 기기 지문 인식 임베딩 모델에 같은 결론이 그대로 옮겨진다고 단정할 수 없다. 모델 구조와 배치정규화 사용 방식이 다르면 PTQ의 취약점도 다르게 나타날 수 있다.

둘째, QAT의 미세조정 학습률과 동결 일정(학습률 $3e-5$, 관찰자 동결 11에폭, 배치정규화 동결 13에폭)은 이 실험에서 한 세트만 시도했다. 하이퍼파라미터를 스왑하면 1.73퍼센트포인트보다 더 낮은 손실 지점이 있을 가능성이 높지만, 이번에는 탐색하지 않았다.

셋째, 미세조정 초반 12에폭의 검증 정확도 진동이 컸다는 사실 자체가 이 스케줄이 최적이지 않을 수 있음을 시사한다. 더 완만한 관찰자 동결 일정이나 더 낮은 초기 학습률이 이 진동을 줄일 수 있는지는 확인하지 않았다.

넷째, fp16 결과에서 보듯 지연 이득은 하드웨어에 강하게 의존한다. 이 논문의 지연 수치는 특정 CPU와 qnnpack 백엔드 조합에서 측정한 것이며, 다른 아키텍처(예를 들어 ARM NEON fp16 네이티브 지원 하드웨어나 GPU)에서는 상대적 순위가 달라질 수 있다.

다섯째, 이 실험은 동적 양자화(Linear 층만 INT8로 바꾸는 경로)도 같은 모델에서 측정했지만 본문 비교에는 포함하지 않았다. 참고로 밝히면, 동적 양자화는 크기를 8.4퍼센트만 줄이면서 지연은 오히려 18퍼센트 늘었다. 이 모델처럼 파라미터 대부분이 컨볼루션에 있고 완전연결 층이 작을 때는 동적 양자화가 실익이 없다는 부정적 결과다.

7 재현

이 실험은 다음 절차로 재현 가능하다.

데이터는 RadioML2016.10a의 6dB SNR 부분집합에서 11개 변조 클래스를 추출하고, 클래스별 균등 분포를 유지한 채 학습/검증/테스트로 나눈다. 테스트 세트는 어떤 학습 단계에도 사용하지 않는다.

모델은 Conv1d(채널 2 to 32 to 64 to 64) 3개 블록(각 블록에 배치정규화와 ReLU 포함), 전역평균풀링, 완전연결 2개 층(64 to 64 to 11)으로 구성한다.

FP32 기준선은 라벨 스무딩 0.05를 적용해 검증 정확도가 개선되지 않으면 20에폭 후 조기종료하는 방식으로 학습한다.

PTQ는 (1) 컨볼루션, 배치정규화, ReLU를 융합하고, (2) 관찰자를 삽입한 뒤 검증 세트로 캘리브레이션하고, (3) qnnpack 백엔드로 정수 커널로 변환한다. 재학습 단계는 없다.

QAT는 (1) 컨볼루션, 배치정규화, ReLU를 학습 가능한 융합 모듈로 묶고, (2) 가중치와 활성화값에 가짜양자화 관찰자를 삽입하고, (3) FP32 학습률의 10분의 1 수준으로 몇 에폭 미세조정하되 후반부에 관찰자를 먼저 동결하고 그다음 배치정규화 이동통계를 동결하고, (4) qnnpack 백엔드로 정수 커널로 변환한다.

측정은 배치 크기 1, CPU 실행 기준으로 30회 워업 후 300회 순전파의 실행 시간을 측정해 p50과 p95를 계산하고, 모델 크기는 직렬화된 파라미터 디렉터리의 바이트 수로 계산한다. 모든 수치는 모델의 자기 보고가 아니라 측정 코드가 직접 계산한 값이어야 한다는 원칙을 지켰다.

8 참고문헌

1. B. Jacob et al., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," arXiv:1712.05877, 2017.
2. R. Krishnamoorthi, "Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper," arXiv:1806.08342, 2018.
3. T. J. O'Shea, T. Roy, and T. C. Clancy, "Over-the-Air Deep Learning Based Radio Signal Classification," IEEE Journal of Selected Topics in Signal Processing, vol. 12, no. 1, 2018 (RadioML2016.10a 데이터셋 출처).
4. PyTorch Quantization 문서, "Quantization Recipe" 및 "(beta) Static Quantization with Eager Mode in PyTorch," pytorch.org/docs.